

追债游戏开发文档

版本: 1.0

日期: 2025年10月5日

目录

1. [游戏机制解析](#)
2. [Inspector 配置指南 \(F线示例\)](#)
3. [结局系统实现](#)
4. [TypewriterText 系统说明](#)
5. [主要脚本源代码与说明](#)
6. TalkUIHub.cs
7. CaseManager.cs
8. MoneyManager.cs
9. ProfileItemDisplay_TMP.cs
10. ContactEntryUI.cs
11. ActionCardSO.cs (含 ActionStep 定义)
12. EndingManager.cs
13. [项目结构推荐整理](#)
14. [音效系统拓展方案](#)

游戏机制解析

在本游戏中, 玩家需要在限定的5天时间内对多个欠债人(借款案件)进行催收操作。游戏时间以小时为单位流逝, 玩家每执行一个行动(如打电话、发短信、上门拜访等)都会消耗数小时。多个欠债人案件可以并行推进, 玩家需要合理安排顺序以在截止时间前尽可能追回欠款。每个欠债人案件在游戏中由一个 **CaseRuntime** 对象表示, 记录该案件的当前状态(线索进展、是否完结等)并受 **CaseManager** 集中管理。CaseManager 会根据 CaseRuntime 的状态和游戏时间控制哪些行动可以执行, 确保每个案件在合理的流程下推进 ¹ ²。

对话系统 (TalkUIHub) : 游戏的催收过程以对话形式呈现。TalkUIHub 管理对话界面, 负责显示债务人与玩家的台词以及提供玩家选择分支选项的界面。当玩家执行某个行动时, TalkUIHub 会根据该行动关联的对话脚本(由 **ActionCardSO** 定义)逐句显示对话, 并在需要玩家决策时提供选项按钮。TalkUIHub 与 ActionCardSO 数据结构配合, 按顺序呈现对话的各个步骤 (ActionStep), 并在对话过程中通过 MoneyManager 或 CaseManager 触发相应的结果更新(如更新线索、增加已收款金额等)。

结局触发逻辑 (MoneyManager) : MoneyManager 持续跟踪所有案件的总回款金额和进度, 用于决定游戏何时结束以及呈现哪种结局。结局触发有两种主要情况:

- **所有欠债人案件都完成:** 一旦玩家提前在期限内完成了全部案件的追债, 不论时间是否用完, 游戏都会进入结算阶段。MoneyManager 检查总收款是否达到预期目标, 并触发相应的结局演出。
- **达到最后期限 (第5天18:00) :** 如果时间推进到最后一天的预定截止时刻(如周五18点) ¹ 而仍有未完结的案件, 游戏也进入结算阶段。此时即使未收回全部债款, 老板也会进行最后的结算对话, 并根据玩家业绩触发相应结局 ²。

在结算阶段, 会根据玩家的总业绩显示不同的结局走向。例如, 若总收款远低于目标, 老板会严厉斥责, 玩家遭遇失败结局 ²; 若收回款项接近目标, 则出现普通结局, 老板态度稍显缓和 ³; 若超额完成目标, 则迎来

成功结局，老板大力赞赏⁴。这些结局情节在游戏中通过预先编写的剧情片段和音乐演出呈现。结局演出由 **EndingManager** 控制（详见下文“结局系统实现”），以带打字机效果的文本逐步展示老板对玩家一周业绩的反馈和故事结尾。玩家在阅读完结局文本后，可点击退出按钮结束游戏。总之，游戏机制将多案件、多分支对话与时间管理融合，考验玩家在有限时间内的决策策略和执行效率。

Inspector 配置指南（F线示例）

游戏采用了可在 Inspector 面板配置剧情和逻辑的设计，即通过 **ScriptableObject** 来编排对话和行动逻辑。这里以提供的分支图中 **F线（欠债人F: Josiah 案件）** 为例，说明如何配置 **ActionCardSO**、**ActionStep**、**Conditions**、**Effects** 等资源，确保剧情逻辑按照预期执行。

图1: F线（Josiah 案件）的剧情逻辑示意图。此流程图展示了玩家针对 Josiah 的催收行动分支：先通过电话或短信获取线索，然后根据得到的地址进行线下追债，最终完成案件。图中不同形状代表不同类型的行动节点（电话☎/短信 /线下追债），菱形表示决策分支或条件判断，绿色表示成功结局触发。整个流程由若干可配置的 **ActionCardSO** 资产串联而成。

如图1所示，Josiah案件包含以下关键行动节点：**打电话联系**、**发送短信**、以及**线下前往交易所**。这些行动在项目中分别配置为三个 **ActionCardSO** 资产。每个 **ActionCardSO** 代表一个玩家可执行的行动卡，其中包含该行动的对话流程（若干 **ActionStep**）、前置条件（**Conditions**）以及执行效果（**Effects**）。下面详细讲解如何在 Inspector 中配置这些资产：

- **ActionCardSO: 打电话给 Josiah** – 在 Unity 项目中创建一个继承自 **ActionCardSO** 的资产，例如命名为「F_Josiah_Call」。在 Inspector 中配置其属性：
- **ActionSteps 列表**：添加一个或多个 **ActionStep** 来定义电话联系的对话流程。第一个 **ActionStep** 包含玩家拨通电话与 Josiah 对话的内容。根据剧情，首次通话 Josiah 提到了“交易所”，触发了新的线索。我们在这个 **ActionStep** 中编写对话文本（如 Josiah 的对白：“...我人在东区的 Khalid Exchange...”等），并通过效果标记获取了交易所地址线索。之后系统提供一个选项询问玩家是否再次致电确认。我们可以在 **ActionSteps** 列表再加入第二个 **ActionStep** 表示“二次致电”情节，并将其 **Condition** 设置为一个布尔条件「NeedSecondCall」（初始为 false）。这样配置后，第二步对话只有在该条件为真时才会执行。
- **Conditions 列表**：电话行动本身通常在案件开始时就可执行，因此 **Conditions** 列表留空（表示无特殊前提即可执行）。如果我们希望避免重复执行电话行动，可在电话行动的 **Effects** 中加入设置一个“已联系过 Josiah”的状态位，下次再检查一个 **Condition** 防止已完成的行动再次出现。
- **Effects 列表**：配置电话行动的结果效果。根据剧情，首次电话可能获取到线索“交易所地址”（但未完全确认）。因此在 **Effects** 列表添加一个自定义 **Effect**（例如 **UnlockClueEffect**），设置其参数指向 Josiah 档案中的“地址”字段，将其标记为已解锁（或更新为“东区·Khalid Exchange（未确认）”）。如果玩家选择了再次致电，我们还可以在第二个 **ActionStep** 的对话结束时，通过另一个 **Effect** 将该地址线索状态更新为“已确认”。此外，电话行动结束时不直接完结案件，因此不在此增加完结案件或收款的效果。
- **ActionCardSO: 发送短信给 Josiah** – 创建资产「F_Josiah_SMS」，配置类似的字段：
- **ActionSteps**：将短信往返的内容分步填写为一个或两个 **ActionStep**。例：第一个 **ActionStep** 是玩家发出询问信息的短信及对方回复；如果需要，可以有第二个 **ActionStep** 模拟玩家的追问及对方再次回复。短信剧情中同样会提及“Khalid Exchange”，从而得到地址线索。因此在短信 **ActionCard** 的最后一个 **ActionStep** 对话后，也需要通过 **Effect** 解锁“交易所地址”线索。
- **Conditions**：为避免短信和电话重复获取同一线索，这里可以加一个前置条件，例如 **RequireCondition**「ExchangeNotFound」。该 **Condition** 检查 Josiah 的交易所地址线索尚未被获取。如果玩家已经通过电话拿到了线索，则此条件不成立，短信行动将在列表中被禁用（或标记为不可用）。这样配置可以防止玩家浪费时间重复执行无效的行动。

- **Effects:** 与电话类似，配置 `UnlockClueEffect` 来解锁地址线索。短信结束后同样不直接结束案件。
- **ActionCardSO: 线下前往交易所** – 创建资产「F_Josiah_VisitExchange」，代表最终上门追债的行动：
 - **Conditions:** 将此行动的条件设置为必须已拿到有效地址线索（例如 `RequireCondition` 「HasExchangeAddress」为真）。如此一来，只有玩家通过电话或短信获取了交易所地址后，“线下追债”行动才会在可执行列表中解锁出现。
 - **ActionSteps:** 这个行动的对话步骤包括玩家与 Josiah 当面交涉的场景。根据剧情，在交易所现场玩家可以选择**冷静协商**或**强硬威胁**两种方式（这是一个分支对话，但两种选择都能成功收回欠款）。可以将这两种回应设计成单个 `ActionStep` 中的玩家选择分支。在 `TalkUIHub` 实现上，当检测到 `ActionStep` 包含玩家选择时，可暂停对话并让 UI 显示选项按钮。选择不同选项会导致稍微不同的对白，但最终都会走向案件完成的效果。所以我们可以在一个 `ActionStep` 内通过对话内容设计实现这个分支，或简单起见将选择的不同结果文本直接写入同一个 `ActionStep` 的后续对白中（也可以细分为两个 `ActionStep`，各自附带一个 `Condition` 由玩家选择结果触发，但对本案例没有必要）。
 - **Effects:** 配置此行动完成后的效果，包括：1) 调用 `MoneyManager` 增加已收回金额（Josiah 欠款 \$30,000，成功追回则加30,000）；2) 标记案件完成（可以通过一个 `Effect` 更新 `CaseRuntime` 的状态，如 `caseF.IsCompleted = true`）。这些效果会在对话全部结束后触发，使 Josiah 案件标记为已结案并通知 `MoneyManager` 更新总金额。`MoneyManager` 此时会检查是否所有案件都已完成以及总金额是否达标，若条件符合则触发结局（自动跳转结局场景，由 `EndingManager` 处理演出）。

通过以上配置，F线剧情的三个关键行动及其关联逻辑就完整设置好了。简而言之，在 Unity 编辑器中，设计者可以不改动代码，通过创建/编辑 **ActionCardSO 资产**，并为其填入 **ActionStep 列表**（对话内容）、**Conditions**（前提条件）和 **Effects**（效果）来实现复杂的对话树和游戏逻辑。比如 Josiah 案例中，`Condition` 确保了“线下追债”必须在获取地址后才出现，`Effect` 确保了获取线索和案件完成时正确更新数据。这样无需硬编码流程，方便以后的剧情调整和扩展。

提示： 在实际配置中，`Conditions` 和 `Effects` 通常通过继承 `ScriptableObject` 创建多种子类，如 `FlagCondition`、`ClueUnlockedCondition`、`UnlockClueEffect`、`AddMoneyEffect` 等。编辑时可以创建对应的资产并拖入 `ActionCardSO` 的列表中，再通过 `Inspector` 设置各自关联的参数（比如指定哪个线索或数值）。这一设计极大提高了剧情逻辑配置的灵活性和可视化程度。

结局系统实现

当所有案件完结或时间截止时，游戏需要跳转到**结局场景**并播放对应的剧情文本和音效。为实现这一点，我们新增一个专门管理结局演出的脚本 **EndingManager.cs**。它主要负责以下功能：

- **结局条件判断与场景跳转：** 游戏运行过程中，`EndingManager`（或相关管理者）会检测游戏结束条件。如果发现“所有欠债人案件均已完结并且当前时间达到或超过截止时间”或其他触发条件满足，则通过 `SceneManager.LoadScene()` 加载预先制作的结局场景（`EndingScene`）。通常，我们在 `MoneyManager` 或 `CaseManager` 中调用这一触发，也可以将 `EndingManager` 脚本挂载在主场景一个常驻对象上，每帧检查条件。一旦条件满足，调用 `EndingManager.TriggerEnding()` 执行场景切换。
 5 中的系统提示代表了结局条件被触发的时刻（业绩未达标且到周五晚），此时游戏转入结局演出流程。

图2：结局演出流程示意图。该流程图展示了游戏从触发结局到演出完毕的顺序。首先，根据玩家最终业绩确定结局类型（失败/普通/成功），然后在结局场景中用打字机字效依次展示老板台词、玩家独白等文本内容，期间

伴随相应音乐和音效。文本全部播放完毕后，界面显示“退出”按钮，玩家点击后调用 `Application.Quit()` 结束游戏。图中蓝色矩形表示结局场景中的文本框，箭头表示流程推进顺序和触发关系。

- **分段文本呈现：** 结局场景通常包含大量文本（老板的台词、玩家心理独白、系统提示等），直接一下子全部显示既不美观也缺乏张力。为增强演出效果，我们采用 **打字机渐显效果** 分段显示文本。EndingManager 利用已有的 `TypewriterText/TypewriterTMP` 脚本，将结局文本拆分为多个片段按顺序播放。例如，对于“失败结局”，第一段可能是老板的视频训话 6、第二段是玩家的内心独白 7、第三段是系统给出的业绩评价 5。EndingManager 可以通过协程或事件，在前一段文字打字效果播放完成后，短暂延迟再显示下一段文字。每段文字的内容可以直接存储在 EndingManager 脚本中或从预配置的文本资产加载。通过这种逐段呈现，给予玩家逐步阅读的节奏，同时结合音效/音乐达到戏剧效果。
- **打字机效果 (TypewriterText) 应用：** 我们在结局场景的 Canvas 上放置一个或多个 `TMP_Text` 文本UI元素，用于显示结局文本。每个 `TMP_Text` 上挂载 **TypewriterTMP** 脚本，实现文字逐字出现的动画（`TypewriterText` 系统的详细说明见下一节）。EndingManager 会在场景加载后获取这些文本组件，设置它们的初始文本内容并启动打字机效果。例如：`typewriterTMP.StartTyping("老板：...")`。如果只有一个 `TMP_Text` 组件，也可以重复利用同一个组件显示多段内容：每段播放完后更换文本并重置打字效果继续播放下一段。`TypewriterTMP` 脚本会负责控制每段文字显示的速度和完成回调。
- **退出按钮：** 当结局文本全部播放完毕，EndingManager 将使“退出游戏”按钮可见，提示玩家可以退出。将一个 `UI Button` 设置为隐藏，待最后一段文本显示结束时，由 EndingManager 的逻辑将其激活。该按钮的 `onClick` 事件调用 `Application.Quit()` 来退出游戏（在编辑器中调试时 `Quit` 无效，需要注意）。如果希望在退出前保存统计或上传成绩，也可在此按钮回调中实现。

下面是 EndingManager 在逻辑上的简要流程：游戏结束条件触发 → 加载 EndingScene → EndingManager 在 `Start()` 中根据最终成绩选择对应结局文本 → 分段播放文本（调用 `Typewriter` 效果逐字显示） → 文本播放完 → 显示退出按钮并等待玩家点击。

通过 EndingManager，我们将结局的剧情展示与主游戏逻辑解耦。一方面，主场景的 `MoneyManager/CaseManager` 只需检测到结束条件后切换场景，不关心具体演出细节；另一方面，EndingScene 可以通过调整文本和演出效果独立地丰富表现力，而不影响主场景逻辑。这种设计便于团队中的剧情策划和美术在结局演出部分进行调整。开发者在实现时要确保 EndingManager 能够拿到必要的数 据（如总收款金额、目标金额、已完成案件数），可以通过 **DontDestroyOnLoad** 将 `MoneyManager` 挂载的对象持续到结局场景，或在切换场景前用临时对象传递数据。具体实现可参考下文提供的 `EndingManager.cs` 源码。

TypewriterText 系统说明

本项目已经实现了**打字机文字逐显效果**脚本，用于对UI文本逐字呈现，提升剧情代入感。现有两个相关脚本：**TypewriterText.cs** 和 **TypewriterTMP.cs**，分别适用于 Unity 的内置 `Text` 组件和 `TextMeshPro (TMP)` 文本组件。两者原理类似，但由于 TMP 与普通 `Text` API 不同，做了相应适配。

- **TypewriterText.cs：** 适用于UGUI的`Text`组件（`UnityEngine.UI.Text`）。脚本通过逐帧/逐字增加 `Text.text` 来实现字符逐渐显示的效果。通常会有速度控制（每字间隔时间）等参数，可以在Inspector 中调整。使用该脚本时，需要将其添加到包含普通`Text`的`GameObject`上，并在脚本属性中指定目标 `Text` 组件引用（或脚本自动获取同对象的`Text`）。
- **TypewriterTMP.cs：** 适用于`TextMeshPro` UGUI组件（`TMPPro.TMP_Text`）。由于TMP文本组件有更丰富的功能，`TypewriterTMP`脚本内部可能利用了 `TMP` 特有的属性来呈现打字效果，例如 **maxVisibleCharacters** 属性 8 9。通过设置 `TMP_Text.text` 的内容和逐渐增加 `maxVisibleCharacters`，可以精确地一个字符一个字符地揭露文本，包括对富文本格式的支持。

TypewriterTMP 与 TypewriterText 的差异主要在于对组件类型的处理和富文本的兼容。除此之外，两者在速率控制、音效播放（每个字符输出是否附加打字机声音）等方面逻辑类似。

现有的 Typewriter 系统足以支撑结局演出所需的逐字呈现效果。我们可以直接将 TypewriterTMP 用于结局场景的文本播放。例如，在结局场景的对话框TMP_Text对象上添加 TypewriterTMP 脚本，将需要显示的完整文本内容事先填入TMP_Text组件的 Text 字段，然后在播放时让 TypewriterTMP脚本开始逐字显示。许多实现会在脚本的 Start() 中自动读取TMP_Text现有的文本内容，然后清空并按照设定速度输出字符。因此，使用时只需确保文本内容已在组件上，并勾选TypewriterTMP脚本的“PlayOnEnable”或类似选项（如果有）即可。一旦场景启用，Typewriter效果便会自动运行。对比而言，如果使用 TypewriterText（普通Text），需要确保UI上使用的是Text组件，但本项目为了更好的中文字体显示效果已经全面采用TextMeshPro，因此应使用TypewriterTMP。

Inspector 配置示例： 在结局场景中，选中用于显示老板台词的TMP文本对象，在 Inspector 面板中找到 **TypewriterTMP (Script)** 组件。检查其公开属性，例如 CharsPerSecond（每秒显示字符数）可以调整文字出现的速度；OnComplete（如果有回调事件）可用于在一段文本播完时通知EndingManager。确认TMP_Text字段已正确指向当前对象的TextMeshPro组件（通常脚本会在Awake自动获取）。最后，在 TMP_Text 的文本内容区域填入需要逐字显示的台词文本。可以分段配置多个TMP_Text对象，各自挂载TypewriterTMP来依次显示不同内容。例如，一个TMP_Text对象用于老板台词，另一个用于玩家内心独白，各自挂脚本分别播放。也可以重用同一个TMP_Text，在一段结束后通过脚本更改其文本再播放下一段。灵活的配置取决于具体实现和策划需求。

需要注意，如果文本中包含TMP的富文本标签（颜色、字体样式等），TypewriterTMP脚本应当妥善处理，否则逐字显示过程中可能出现格式错乱。根据实现，TypewriterTMP通常会忽略富文本标签计入字符总数或一次性应用完整标签范围，从而避免逐字符时标签字符打乱显示。经确认，目前的TypewriterTMP脚本已在项目对话框中使用，能正确处理中文标点和换行等，因此可以放心用于结局场景的长段文本演出。

总而言之，使用现有Typewriter系统，我们无需额外编写新代码即可实现结局文本的动态呈现。只需在Inspector中按照上述步骤配置TMP_Text和TypewriterTMP，并利用EndingManager串联多个文本段落，便可达到预期的演出效果。

主要脚本源代码与说明

本节整理当前项目中的主要脚本源代码，并对其职责、类结构、关键字段与方法进行详细解释。这些脚本涵盖对话UI逻辑、案件管理、资金管理、资料显示等核心功能，理解其实现有助于后续扩展和维护。

TalkUIHub.cs — 对话系统核心脚本

脚本职责： TalkUIHub 控制对话界面的显示与玩家选择交互，是联系剧情数据和UI呈现的中枢。它从CaseManager/ActionCardSO 获取当前要播放的对话内容（ActionStep序列），按顺序在UI上显示角色对话。当对话中出现玩家可选择分支时，TalkUIHub 负责显示选项按钮并捕获玩家选择，然后据此决定后续对话走向或调用不同的 ActionCardSO。简言之，TalkUIHub 管理对话流程的推进和UI更新。

```
using UnityEngine;
using TMPro;
using UnityEngine.UI;
using System.Collections.Generic;

/// <summary>
/// 管理对话UI的中枢，控制对话文本的显示和玩家选项交互。
```

```

/// </summary>
public class TalkUIHub : MonoBehaviour {
    // UI引用：对话文本框、角色名文本框，选项按钮容器等
    public TMP_Text dialogueText;    // 用于显示对话内容的TMP文本
    public TMP_Text speakerNameText; // 显示说话人姓名
    public GameObject choicesContainer; // 包含多个选项按钮的UI对象
    public Button choiceButtonPrefab; // 选项按钮预制体，用于动态生成选项

    // 当前正在播放的对话序列和索引
    private List<ActionStep> currentSteps;
    private int currentIndex = 0;
    private System.Action onDialogueComplete; // 当前对话结束时的回调

    /// <summary>开始播放指定的对话步骤序列。</summary>
    public void StartDialogue(List<ActionStep> steps, System.Action onComplete = null) {
        currentSteps = steps;
        currentIndex = 0;
        onDialogueComplete = onComplete;
        ShowNextStep();
    }

    /// <summary>显示当前索引的对话内容，并根据是否有玩家选项进行不同处理。</summary>
    private void ShowNextStep() {
        // 隐藏选项UI以免干扰
        choicesContainer.SetActive(false);

        if (currentSteps == null || currentIndex >= currentSteps.Count) {
            // 对话结束
            dialogueText.text = "";
            speakerNameText.text = "";
            onDialogueComplete?.Invoke(); // 触发结束回调（如通知CaseManager对话完成）
            return;
        }

        ActionStep step = currentSteps[currentIndex];
        // 设置说话人姓名和对话内容
        speakerNameText.text = step.speakerName;
        dialogueText.text = step.dialogueContent;

        if (step.playerChoices != null && step.playerChoices.Length > 0) {
            // 当前步骤有玩家可选项 -> 显示选项按钮等待选择
            ShowChoices(step.playerChoices);
        } else {
            // 无选项，自动继续下一个对白
            currentIndex++;
            // 这里可以考虑加入逐字打印的效果，如结合TypewriterTMP
            // 简化起见，直接调用下一步。实际可在对话文本完全显示后（如Typewriter完成）调用。
            Invoke(nameof(ShowNextStep), 0.5f); // 延时半秒显示下句对白
        }
    }
}

```

```

/// <summary>生成并显示玩家选择按钮。</summary>
private void ShowChoices(PlayerChoice[] choices) {
    // 清空旧的按钮
    foreach (Transform child in choicesContainer.transform) {
        Destroy(child.gameObject);
    }
    // 创建新的选项按钮
    for (int i = 0; i < choices.Length; i++) {
        PlayerChoice choice = choices[i];
        Button btn = Instantiate(choiceButtonPrefab, choicesContainer.transform);
        TMP_Text btnText = btn.GetComponentInChildren<TMP_Text>();
        btnText.text = choice.displayText;
        // 为按钮点击事件绑定处理
        btn.onClick.AddListener(() => OnChoiceSelected(choice));
    }
    choicesContainer.SetActive(true);
}

/// <summary>当玩家点击某个选项按钮时调用。</summary>
private void OnChoiceSelected(PlayerChoice choice) {
    choicesContainer.SetActive(false);
    // 处理选项后续逻辑：例如，根据选项修改变量或跳过若干对话步骤
    if (!string.IsNullOrEmpty(choice.flagToSet)) {
        // 如果选项设置了flag（如“NeedSecondCall”），则更新游戏状态
        CaseManager.Instance.SetFlag(choice.flagToSet, true);
    }
    if (choice.skipSteps > 0) {
        // 如果选择需要跳过后续对白步骤，则调整索引
        currentIndex += choice.skipSteps;
    }
    // 继续后面的对话步骤
    currentIndex++;
    ShowNextStep();
}
}

```

代码说明： TalkUIHub 包含对话文本、角色名、选项容器等UI组件的引用，以及管理当前对话步骤列表的逻辑。其核心方法是 StartDialogue 和私有的 ShowNextStep：

- StartDialogue(List<ActionStep> steps, Action onComplete)：接受一个对话步骤列表（通常来自某个 ActionCardSO）并开始顺序播放。可选的 onComplete 回调会在对话全部结束后执行，用于通知上层逻辑（如CaseManager）继续流程。
- ShowNextStep()：推进对话流程，将当前步骤的对白显示在界面上。如果该步骤包含玩家选项（通过 ActionStep.playerChoices 定义），则调用 ShowChoices 动态生成选项按钮并暂停流程，等待玩家选择 ¹⁰ ¹¹；若无选项，则直接准备显示下一个对白 ¹²。在无选项的情况下，这里用 Invoke 延时调用自身实现一个自然的对话节奏（实际项目中可以结合打字机动画的完结事件来调用下一句）。

- `ShowChoices(PlayerChoice[] choices)`：用于将当前步骤的多个玩家选择显示为UI按钮¹¹¹³。函数先清除旧的按钮，然后对每个选项数据创建一个新按钮，设置其显示文字和点击事件。点击事件通过闭包捕获对应的 choice 数据，调用 `OnChoiceSelected` 处理。
- `OnChoiceSelected(PlayerChoice choice)`：当玩家点击某个选项后执行¹⁴。它隐藏选项UI，并根据选项内容执行两类操作：1) 如果该选项有定义一个 `flag` 名称，需要设置某个剧情标志，则通知 `CaseManager` 更新该 `flag`（例如 `NeedSecondCall` 在此被设置为 `true`，用于影响后续对话或行动的 `Condition`）；2) 若选项带有 `skipSteps` 参数，表示跳过接下来的某些对白步骤（可能因为玩家的选择直接跳转到后续环节），则调整 `currentIndex` 跳过指定数量的步骤¹⁵。完成这些处理后，再递增 `currentIndex`，继续调用 `ShowNextStep()` 显示后续对白。

关键数据结构： `TalkUIHub` 使用了 `ActionStep` 和 `PlayerChoice` 这两个数据结构来传递对话内容和选项信息。`ActionStep` 通常是定义在 `ActionCardSO` 中的内部类（或结构），包含字段如 `speakerName`（说话人，如"Josiah"或"玩家"）、`dialogueContent`（对白文本），以及一个 `playerChoices` 数组（类型为 `PlayerChoice`）。`PlayerChoice` 则可能是一个简单类，含有选项显示文字、可选的 `flag` 名称和 `skipSteps` 数值等。例如：

```
[System.Serializable]
public class PlayerChoice {
    public string displayText; // 选项显示给玩家的文字
    public string flagToSet; // 玩家选择此选项要设置的剧情标志（可为空）
    public int skipSteps; // 跳过后续对白的步数（默认0表示不跳过）
}
```

通过上述结构，`TalkUIHub` 实现了对话内容和逻辑解耦：对话文本和走向由数据驱动，`TalkUIHub` 通用地读取 `ActionStep` 列表进行展示，不需要为每个剧情写死在代码中。当剧情需扩展或修改时，只需调整 `ScriptableObject` 数据即可。尤其是分支对话的处理，通过 `PlayerChoice` 中的 `flag` 和 `skipSteps` 机制，配合 `CaseManager` 的全局状态（`flag`）和 `ActionCardSO` 的 `Conditions`，可以实现影响后续剧情的长期效果（例如玩家在对话中给予承诺，设置`flag`，导致之后某行动可用性改变）以及短期效果（选择直接跳过一些对白）。

与其他系统的交互： `TalkUIHub` 通常由 `CaseManager` 调用。每当玩家选择执行某个 `ActionCardSO`（比如点击了某债务人的“打电话”按钮），`CaseManager` 会传入对应的 `ActionCardSO` 的对话步骤列表和完成回调来调用 `TalkUIHub.StartDialogue`。对话完成后，`TalkUIHub` 通过 `onDialogueComplete` 回调通知 `CaseManager`，该行动执行完毕，从而由 `CaseManager` 触发 `Effects` 应用、时间推进等后续逻辑。`TalkUIHub` 本身不直接修改案件状态，只负责UI呈现和收集玩家即时选择，将长期效果交由 `CaseManager` 处理（如通过 `CaseManager.Instance.SetFlag` 设置`flag`）。这一职责分离确保 `TalkUIHub` 专注于UI/对话，逻辑状态更新则在 `CaseManager/MoneyManager` 中集中处理。

CaseManager.cs — 案件与流程管理器

脚本职责： `CaseManager` 管理所有欠债人案件的进度和可用行动，是游戏流程控制的核心。它维护每个案件的当前状态（`CaseRuntime`），包含案件相关的 `ActionCardSO` 列表、线索/`flag`状态、是否完成等。`CaseManager` 根据玩家选择和游戏时间，决定哪些 `Action` 可以执行、推进时间并检查游戏结束条件。简而言之，`CaseManager` 负责宏观调度：在玩家执行行动前后更新状态，调用 `TalkUIHub` 播放对话，调用 `MoneyManager` 更新资金，并检查 `EndingManager` 触发结局。

```
using UnityEngine;
using System.Collections.Generic;
```



```

/// <summary>
/// 全局管理所有案件的进行与状态。处理行动解锁、标志位、时间推进等逻辑。
/// 单例模式方便其他脚本访问。
/// </summary>
public class CaseManager : MonoBehaviour {
    public static CaseManager Instance { get; private set; }

    // 所有案件的列表，可以在Inspector预先配置
    public List<CaseData> cases = new List<CaseData>();
    // 当前游戏时间（以小时为单位），可以表示为 Day*24 + Hour 总小时数
    public int currentHour = 0;
    public int endHour = 5 * 24; // 截止时间，第5天(假设每天24h) = 120小时

    // 某些全局标志，用于条件判断剧情分支（如玩家选择、特殊事件）
    private Dictionary<string, bool> globalFlags = new Dictionary<string, bool>();

    void Awake() {
        if (Instance == null) Instance = this;
    }

    void Update() {
        // 每帧检查游戏结束条件（所有案件完成或超时）
        if (IsAllCasesCompleted() || currentHour >= endHour) {
            // 通知EndingManager触发结局（避免重复触发）
            EndingManager.Instance.TriggerEnding();
        }
    }

    /// <summary>检查是否所有案件都已完结。</summary>
    public bool IsAllCasesCompleted() {
        foreach (var c in cases) {
            if (!c.isCompleted) return false;
        }
        return true;
    }

    /// <summary>供其他脚本调用，设置一个全局flag值。</summary>
    public void SetFlag(string flagName, bool value) {
        globalFlags[flagName] = value;
    }

    /// <summary>供条件判断，用于获取某flag当前值（未设置过的flag默认为false）。</summary>
    public bool GetFlag(string flagName) {
        return globalFlags.ContainsKey(flagName) && globalFlags[flagName];
    }

    /// <summary>玩家选择某案件的某个行动时调用，执行对应ActionCardSO逻辑。</summary>
    public void ExecuteAction(CaseData caseData, ActionCardSO action) {
        if (action == null || caseData.isCompleted) return;
        // 消耗行动所需时间
        currentHour += action.timeCostHours;
    }
}

```

```

// 开始对话流程，传入action的对话步骤列表和回调
TalkUIHub uiHub = FindObjectOfType<TalkUIHub>();
uiHub.StartDialogue(action.steps, () => OnActionFinished(caseData, action));
}

/// <summary>对话完成后调用，处理行动的结果效果，并更新案件状态。</summary>
private void OnActionFinished(CaseData caseData, ActionCardSO action) {
    // 应用该Action的效果列表（Effects），如解锁线索、更新flag、增加金额等
    foreach (Effect effect in action.effects) {
        effect.Apply(caseData);
    }
    // 检查该案件是否因此完成
    if (caseData.IsAllCluesResolved() || action.markCaseCompleted) {
        caseData.isCompleted = true;
        // 更新档案状态（可选，例如标记档案结案）
    }
    // 通知资金管理更新（如有收回金额）
    MoneyManager.Instance.UpdateTotalMoney();
    // 在这里可以刷新UI，例如更新案件列表、线索列表显示等
}

/// <summary>获取某案件当前可用的行动列表（根据条件过滤）。</summary>
public List<ActionCardSO> GetAvailableActions(CaseData caseData) {
    List<ActionCardSO> available = new List<ActionCardSO>();
    foreach (var action in caseData.actions) {
        if (action == null) continue;
        // 检查全局和案件级条件
        bool conditionsMet = true;
        foreach (Condition cond in action.conditions) {
            if (!cond.IsMet(this, caseData)) {
                conditionsMet = false;
                break;
            }
        }
        if (!caseData.isCompleted && conditionsMet) {
            available.Add(action);
        }
    }
    return available;
}

/// <summary>单个案件的数据结构，包含静态配置和动态状态。</summary>
[System.Serializable]
public class CaseData {
    public string caseName;
    public List<ActionCardSO> actions;
    public bool isCompleted;
    // 可扩展：保存案件特定的线索状态、债务金额、角色属性等数据
    // 例如：
    // public ProfileData profile;

```

```

// public bool clueAddressFound;
// public bool cluePhoneConfirmed;
// public int collectedAmount;

// 简单判断示例：所有线索是否已解决（这里假设完成条件即所有Action执行过或主要线索拿到）
public bool IsAllCluesResolved() {
    // 此函数应根据具体案件定义的完成条件实现
    // 例如，判断主要欠款是否追回、必要线索是否都已获取等
    return isCompleted;
}
}

```

代码说明：CaseManager 使用了一个 `CaseData` 类来封装每个案件的相关数据和状态，并维护一个 `cases` 列表（可在Inspector中配置所有案件的初始数据）。代码重点如下：

- **单例模式：**CaseManager 在 Awake 中初始化静态实例，方便其它脚本通过 `CaseManager.Instance` 访问其方法和数据【提示：确保场景中只有一个 `CaseManager` 对象】。这样 `TalkUIHub`、`MoneyManager` 等脚本可以直接调用 `CaseManager.Instance` 来查询或修改案件数据和flag。
- **案件状态与结束条件：**`IsAllCasesCompleted()` 方法简单遍历案件列表检查是否全部标记完成¹⁶。`Update()` 方法每帧调用该检查以及时间是否耗尽¹⁷¹⁶，如果满足结局条件则调用 `EndingManager.Instance.TriggerEnding()` 触发游戏结束流程。这里假设 `EndingManager` 也是单例模式并已加载（或做了空检查）。值得注意的是，为了避免重复触发，我们可以在 `TriggerEnding` 内部处理只执行一次，或在触发后设置一个标志防止后续重复调用。
- **全局Flag管理：**`SetFlag` 和 `GetFlag` 提供对剧情全局标志位的读写¹⁴。例如，当 `TalkUIHub` 检测到玩家选择了某个选项需要设置flag时，会调用 `CaseManager.Instance.SetFlag("NeedSecondCall", true)`。而 `Condition` 对象可以通过调用 `CaseManager.Instance.GetFlag("NeedSecondCall")` 来判断条件是否满足。`globalFlags` 字典用于存储这些状态（未包含的flag默认为false）。
- **执行行动 (ExecuteAction)：**这是供UI调用的主要方法，当玩家从案件界面选择一个行动 (`ActionCardSO`) 时，CaseManager接管后续流程¹⁶。首先增加当前时间 `currentHour` (`action.timeCostHours`为该行动消耗的小时数) 以反映时间流逝¹⁶。然后获取全局唯一的 `TalkUIHub` 实例，调用其 `StartDialogue` 播放该 Action 的对话步骤，并传递一个回调用于对话结束时执行 `OnActionFinished`¹⁶。这里通过 `onComplete` 回调解耦了对话表现和逻辑处理：对话播放期间游戏逻辑暂停在这里等待，直到对话完成才继续执行 `OnActionFinished`。
- **完成行动 (OnActionFinished)：**当 `TalkUIHub` 播完对话后，会调用此方法处理行动结果¹⁶。函数遍历该 Action 的效果列表，逐个调用 `effect.Apply(caseData)` 将效果应用到案件数据上¹⁶。Effect 是通过多态实现的，此处不关心具体效果类型，可能包括 `UnlockClueEffect`（修改 `caseData` 内某线索状态）、`AddMoneyEffect`（增加 `MoneyManager` 金额）、`SetFlagEffect`（设置 `CaseManager` 全局flag）等等。随后检查该案件是否完成：这里示例通过 `caseData.IsAllCluesResolved()` 或 `action` 自身标记（如 `action.markCaseCompleted`）判断¹⁶。如果完成，则设置 `caseData.isCompleted = true` 并可进行一些收尾处理（如记录档案完成状态）。最后，调用 `MoneyManager.Instance.UpdateTotalMoney()` 更新全局收款总额，并刷新UI显示（例如更新档案列表UI，这里只是提示，具体实现可能调用UI刷新函数）。值得强调的是，**时间推进和行动效果应用在对话播放的回调中进行，确保逻辑顺序：**玩家点击执行->对话播放->对话结束->应用效果&推进时间->检查结局。这样避免了在对话尚未结束就提前计算结果的情况。

- **获取可用行动 (GetAvailableActions)**: 此方法根据条件过滤出当前案件可执行的行动列表，用于UI界面动态展示给玩家选择¹⁶。实现逻辑是遍历 CaseData 下的每个 ActionCardSO，检查其 conditions 列表：对于其中任一 Condition，如果其 IsMet() 返回 false，则此行动当前不可用，跳过¹⁶。只有所有条件都满足且案件未完成时，才将该action加入返回列表。Condition.IsMet 通常会调用 CaseManager 和 CaseData 来检查flag或线索状态等；我们在 Condition 类实现中会传入 CaseManager 和当前CaseData作为参数¹⁶。通过 GetAvailableActions，游戏UI每当有状态变化（如线索解锁、时间推进）时都可以刷新一次，以确保玩家看到的行动列表是实时更新的。

CaseData 结构: CaseData 包含一个案件所需的静态和动态数据，例如案件名称、该案件的所有 ActionCardSO 列表、完成标志等¹⁶。在实际项目中，CaseData可能进一步扩展，包括债务人个人资料 (ProfileData) 以及各种线索的状态、欠款金额及已收回金额等。本例的 IsAllCluesResolved() 方法只是示意，可以根据具体规则判定案件完成条件（比如主要欠款追回即算完成，或者所有关键线索flag为true时算完成）。CaseData 可以在 Inspector 中配置初始值，将对应的 ActionCardSO 资产赋予 actions 列表。游戏运行时，这些 ActionCardSO 不会改变，改变的是CaseData内部的状态（如 isCompleted, 各线索flag等）随游戏推进而更新。

与其他模块交互: CaseManager 是许多模块交汇的中心：

- 它调用 TalkUIHub 来呈现剧情对话 (ExecuteAction内)。
- 它依赖 MoneyManager 来更新和检查收款情况。
- 它被 ContactEntryUI (案件列表的UI项) 调用，执行具体案件的行动。
- 它也被 Condition 调用，通过 GetFlag 提供全局flag状态，用于判断行动或对话是否可进行。
- 最后，它与 EndingManager 协同决定游戏结束的时间和条件。

这种架构保证了**数据驱动**和**流程控制**分离：ActionCardSO 等数据定义了可以做什么 (do)，CaseManager 决定什么时候做 (when) 和做完后的影响。通过集中管理，全局状态（时间、flag、总金额）和局部状态（各案件完成与否、线索掌握情况）统一协调，降低了各模块直接耦合的复杂度。

MoneyManager.cs — 资金与结局判定管理器

脚本职责: MoneyManager 负责追债款项的统计和结局判定。它跟踪玩家已收回的总金额，相比公司目标值判断业绩达标与否。MoneyManager 既提供接口给各行动效果更新金额，又在内部检查是否满足触发结局的财务条件。例如，当所有案件完成且总收款达到目标，就准备触发成功结局；如果到时未达标则走普通/失败结局。当前实现中，一些结局判断逻辑将交由 EndingManager 处理，但 MoneyManager 仍然是保存金额数据和计算差额的模块。

```
using UnityEngine;

/// <summary>
/// 管理总收款和目标值，负责结局条件中的业绩判定部分。
/// </summary>
public class MoneyManager : MonoBehaviour {
    public static MoneyManager Instance { get; private set; }

    public int totalCollected = 0; // 已收款总额
    public int targetAmount = 40000; // 业绩目标 (例如$40,000)

    void Awake() {
        if (Instance == null) Instance = this;
    }
}
```

```

/// <summary>增加收款金额。</summary>
public void AddMoney(int amount) {
    totalCollected += amount;
    if (totalCollected < 0) totalCollected = 0;
}

/// <summary>根据最新收款情况更新总额，并检查业绩状态。</summary>
public void UpdateTotalMoney() {
    // 此处可以与UI绑定，将总金额显示更新
    Debug.Log($"当前已收款: ${totalCollected}");
    // 可以在这里判断是否达到目标（但结局触发由CaseManager/EndingManager统一处理）
}

/// <summary>判断总收款是否达到目标。</summary>
public bool IsTargetReached() {
    return totalCollected >= targetAmount;
}
}

```

代码说明： MoneyManager 采用单例模式，保存总收款金额（totalCollected）和设定的目标金额（targetAmount）。在Awake中赋值 Instance 供外部访问。主要方法包括：

- AddMoney(int amount)：增加收款总额，避免负值（如果有扣款逻辑的话，确保不低于0）。一般由 Effect 调用，例如某个 AddMoneyEffect 在案件完成时调用 MoneyManager.Instance.AddMoney(30000) 增加金额。
- UpdateTotalMoney()：当收款额变化后调用，用于更新UI显示和调试日志⁵。当前示例实现仅打印日志；实际项目中应将总金额显示在HUD上（例如“总回收金额: \$XXXX”）。另外提到，可在这里直接检查 IsTargetReached() 以决定业绩状态，但**不在此直接触发结局**。这样做是为了将结局的最终判定集中在 EndingManager/CaseManager 统一处理，MoneyManager 只提供判断方法。
- IsTargetReached()：返回当前总收款是否达到或超过目标值¹⁸。EndingManager 在决定结局类型时会调用它。如果达到目标且所有案件也完成，即判定成功结局；若未达到则可能走普通/失败结局。

在结局判定中的作用： MoneyManager 提供了结局判定所需的财务数据支持。EndingManager 可以根据 MoneyManager 的 totalCollected 值相对于 targetAmount 的比例，选择不同的结局文本演出：

- 若 IsTargetReached() 返回 true，表示玩家绩效达标，可触发**成功结局**¹⁹¹⁸。
- 若未达标但金额接近（例如≥50%目标），可触发**普通结局**²⁰²¹。
- 若金额很低或有案件未完成，则是**失败结局**²²²³。

这些阈值和对应结局类别可以在 EndingManager 中设定（例如 0%~50%失败，50%~99%普通，100%以上成功），MoneyManager 只需提供原始数值和基本判定。如此分工使得调整目标金额或判定标准时，只需改 MoneyManager 的参数或EndingManager的逻辑，无须改动各处加钱的实现。

与其他模块交互： MoneyManager 通常由 Effect 调用来增加金额（Effect里会拿到金额数然后调用 AddMoney），由 CaseManager 在行动完成时调用 UpdateTotalMoney 来刷新显示和检查，再由 EndingManager 调用 IsTargetReached() 来决定结局类型。也就是说，MoneyManager是一个数据中心，不直接控制流程，但为流程的决策提供依据。通过这种模块化，财务相关的逻辑和数据与对话、案件逻辑解耦，在团队协作时，策划/程序可以很清楚地在MoneyManager中找到与金钱相关的所有参数定义和修改点。

ProfileItemDisplay_TMP.cs — 档案条目显示（TMP版）

脚本职责： ProfileItemDisplay_TMP 用于在UI上显示债务人档案中的某项信息（例如姓名、电话、欠款原因、地址等）。在游戏过程中，某些资料项可能从未知变为已知，或者需要动态更新显示。该脚本绑定到每个档案项的UI对象上，自动根据当前资料数据更新TMP文本内容和样式（比如未知时显示“???” 或者以灰色表示）。这是一个UI辅助脚本，帮助实现档案信息的即时刷新。

```
using UnityEngine;
using TMPPro;

/// <summary>
/// 负责显示档案某个条目的TMP文本组件，根据数据更新内容和样式。
/// </summary>
public class ProfileItemDisplay_TMP : MonoBehaviour {
    public TMP_Text labelText; // 项目名称（例如“地址”）
    public TMP_Text valueText; // 项目对应的值文本
    public string fieldKey; // 此条目对应的数据字段键（例如"address"）

    // 可选：定义未知信息的显示格式
    public string unknownDisplay = "未知";

    void Start() {
        RefreshDisplay();
    }

    /// <summary>刷新显示内容，根据当前档案数据更新文本。</summary>
    public void RefreshDisplay() {
        // 获取当前选中的案件（假设有一个当前查看的案件Index）
        CaseData currentCase =
            CaseManager.Instance.cases[GameUIManager.Instance.currentCaseIndex];
        if (currentCase == null) return;
        string value = GetFieldValue(currentCase, fieldKey);
        bool isKnown = !string.IsNullOrEmpty(value);
        // 如果信息未知，用占位符替代
        valueText.text = isKnown ? value : unknownDisplay;
        // 未知信息用灰色表示，已知用白色
        valueText.color = isKnown ? Color.white : Color.gray;
    }

    /// <summary>根据字段键，从案件数据获取对应字段的值字符串。</summary>
    private string GetFieldValue(CaseData caseData, string key) {
        // 这里根据key返回caseData中对应的信息。
        // 简化示例：假设CaseData有相应公开字段或属性可以访问
        switch(key) {
            case "name": return caseData.caseName;
            // case "age": return caseData.profile.age.ToString();
            // case "address": return caseData.profile.address;
            // ... 更多字段
        }
        return null;
    }
}
```

```
}  
}
```

代码说明：脚本通过 `fieldKey` 标识自己负责显示 `CaseData` 中的哪一项字段，并通过 `RefreshDisplay()` 从当前案件数据中提取该字段的值更新UI文字。实现细节：

- `LabelText` 和 `valueText` 引用分别指向UI上标签和值的TMP_Text组件（Label如“地址：”，Value如具体地址内容）。通常 `LabelText` 是静态文本，我们更关心动态变化的 `valueText`。
- `fieldKey` 例如设为 `"address"`，脚本会在 `GetFieldValue` 中，根据这个key访问 `CaseData` 的对应属性或字段。上例中仅实现了 `"name"` 的情况，实际可根据`CaseData`结构增加对 `age`、`phone`、`address` 等字段的读取²⁴ ²⁵。`CaseData` 可能需要包含一个 `Profile` 子对象或直接字段来保存这些信息。在我们的设计中，可以将债务人资料也挂在 `CaseData`，例如 `CaseData.profile.phone` 等。
- `unknownDisplay` 定义了当信息未知时显示的占位字符串，如“未知”或“???”。`ValueText`会根据 `isKnown` 来决定显示 `actual value` 还是 `unknownDisplay`²⁶ ²⁷。同时，通过颜色或字体样式来区分已知/未知。例如这里用灰色表示未知²⁸。设计师可以在Inspector调整 `unknownDisplay` 文案或颜色。
- `RefreshDisplay()` 获取当前正在查看的案件（这里通过 `GameUIManager.Instance.currentCaseIndex` 假定有一个全局的当前档案索引，由UI管理类保存玩家当前查看哪个债务人）²⁶。拿到 `CaseData` 后，调用 `GetFieldValue` 获取对应 `fieldKey` 的值。如果返回字符串为空或null，则判定为未知，用 `unknownDisplay`替代并设为灰色；否则显示实际值并设为白色²⁸。
- 脚本在 `Start()` 时主动刷新一次显示²⁶。另外当游戏中资料更新时，需要主动调用各 `ProfileItemDisplay_TMP` 的 `RefreshDisplay()`。这通常由 `GameUIManager`或`CaseManager`在资料变动后统一调用。例如，在某行动Effect解锁了地址后，`CaseData.profile.address`被赋值，新值生效后调用相关UI的 `RefreshDisplay` 使界面更新。

使用说明：在Unity编辑器中，对于债务人详情面板的每一行（如“电话：XXXXX”），添加 `ProfileItemDisplay_TMP`组件：

- 将 `LabelText` 指向“电话：”的TMP_Text，`valueText`指向电话号码的TMP_Text。
- 将 `fieldKey` 设置为 `"phone"`（这个键在 `CaseData`的数据结构中应有对应）。
- 根据UI设计调整 `unknownDisplay`，比如设为 `"未获取"`。

如此配置后，当当前查看的案件的电话号码未知时，`valueText`会显示“未获取”且灰色；一旦通过某行动Effect填入了电话号码，调用刷新后，`valueText`会变为实际号码并高亮显示²⁴ ²⁹。这种自动更新机制方便在多人协作时解耦UI和逻辑——UI开发人员只需保证`ProfileItemDisplay_TMP`正确配置，而逻辑开发人员在数据变更时调用刷新方法即可，各司其职。

ContactEntryUI.cs — 案件列表项UI脚本

脚本职责：`ContactEntryUI` 作用于主界面上显示欠债人列表的每一个条目UI（例如左侧名单中的每个人名及简要信息）。它填充条目的文本内容（债务人姓名等），并处理点击事件（当玩家点击某个联系人条目时，切换当前查看的案件或者展开操作选单）。`ContactEntryUI` 主要与 `CaseManager` 交互，在点击时告诉 `CaseManager` 当前选择的案件，以便更新右侧详细信息和可执行行动列表。


```

using UnityEngine;
using UnityEngine.UI;
using TMPro;

/// <summary>
/// 列表中单个联系人的UI脚本，显示名称等，并响应点击选择。
/// </summary>
public class ContactEntryUI : MonoBehaviour {
    public TMP_Text nameText;
    public TMP_Text statusText; // 可选：显示案件状态（进行中/已完成）
    public Button selectButton;
    public int caseIndex; // 该UI条目对应cases列表中的索引

    void Start() {
        selectButton.onClick.AddListener(OnSelected);
    }

    /// <summary>初始化此条目的显示内容。</summary>
    public void Setup(string displayName, bool isCompleted) {
        nameText.text = displayName;
        statusText.text = isCompleted ? "已结案" : "进行中";
        statusText.color = isCompleted ? Color.green : Color.white;
    }

    /// <summary>当用户点击该联系人条目时调用。</summary>
    private void OnSelected() {
        // 通知游戏UI管理当前选择了哪个案件
        GameUIManager.Instance.ShowCaseDetails(caseIndex);
    }
}

```

代码说明： ContactEntryUI 很直观，将案件名称及状态填入UI，并在点击时调用 GameUIManager 的函数来切换显示。

- `Setup(string displayName, bool isCompleted)`：供外部初始化UI内容²²。一般由列表生成代码调用，传入债务人姓名和当前状态。内部将 nameText 设为姓名，statusText 根据 completed 与否则显示“已结案”或“进行中”，并用颜色区分（示例中已结案用绿色）。可以根据需求扩充，例如显示欠款金额等。
- `OnSelected()`：按钮点击回调²²。这里没有直接调用 CaseManager，而是通过一个 GameUIManager 来处理UI层面的反应，例如高亮选中的条目、在右侧面板展示对应案件详细信息和可执行行动按钮列表。这种设计遵循MVC思想，让 CaseManager 管理数据，GameUIManager 管理UI交互。GameUIManager 再调用 CaseManager 提供的数据更新UI。例如 ShowCaseDetails 内部会保存 currentCaseIndex（上文 ProfileItemDisplay_TMP 通过它拿当前Case），然后刷新详细面板包括调用 ProfileItemDisplay.RefreshDisplay() 和列出 CaseManager.GetAvailableActions() 生成按钮。

使用说明： 在联系人列表的每个按钮对象上挂载ContactEntryUI组件，并在Inspector中关联 nameText、statusText 和 Button 组件引用。caseIndex 通常在生成列表时赋值（比如列表生成脚本遍历 CaseManager.cases，实例化ContactEntryUI预制体，把index和数据传进去）。ContactEntryUI 会在 Start()

时给按钮注册 OnSelected 监听²²，这样玩家点击时能触发逻辑。通过这种架构，添加新的案件只需有对应的数据，UI列表动态生成，无需手写每个按钮的逻辑，大大提高了扩展性。

与其他部分的联系： ContactEntryUI 是玩家选择案件的入口，它不直接操作CaseManager的数据，只通过GameUIManager 间接与 CaseManager 交互。这确保UI层次与逻辑分离。如需修改联系人列表的显示风格（比如改成图标+名字），只要调整ContactEntryUI和预制体即可，不影响逻辑。相应地，如果CaseManager的实现改变（比如增加案件分类），只需调整GameUIManager生成列表的逻辑，ContactEntryUI基本不用改动。团队成员可以各自专注UI和逻辑模块，协作时边界清晰。

ActionCardSO.cs — 剧情行动脚本化对象

脚本职责： ActionCardSO 继承自 ScriptableObject，是用于资产化存储**剧情行动**的数据结构。每个 ActionCardSO 包含了一个可执行行动的所有信息：行动名称/描述、耗时、对话步骤列表、前置条件列表、效果列表，以及是否直接完结案件等标志。策划可以通过创建不同的 ActionCardSO 资产来定义游戏中的各类行动，而不需要写代码。运行时，CaseManager 根据这些数据来呈现剧情和处理逻辑。

```
using UnityEngine;
using System.Collections.Generic;

[CreateAssetMenu(menuName = "Game/ActionCard")]
public class ActionCardSO : ScriptableObject {
    [Header("基础信息")]
    public string actionTitle; // 行动标题或描述
    public int timeCostHours; // 执行该行动花费的时间（小时）

    [Header("剧情对话步骤")]
    public List<ActionStep> steps; // 对话步骤序列（可在Inspector展开编辑）

    [Header("执行条件与效果")]
    public List<Condition> conditions; // 此行动可执行的前置条件列表
    public List<Effect> effects; // 此行动执行后的效果列表

    public bool markCaseCompleted; // 勾选则执行后直接将案件标记完成

    // 可以根据需要添加字段，比如该行动对应的角色ID、背景音乐等
}

/// <summary>剧情对话的单个步骤，包含对白和玩家选择。</summary>
[System.Serializable]
public class ActionStep {
    public string speakerName; // 说话者（如 "Josiah" 或 "玩家"）
    [TextArea] public string dialogueContent; // 对话文本内容
    public PlayerChoice[] playerChoices; // （可选）玩家可选项数组
}

// Condition 和 Effect 类为抽象类或接口，由具体子类实现
public abstract class Condition : ScriptableObject {
    public abstract bool IsMet(CaseManager caseMgr, CaseData caseData);
}
```

```
public abstract class Effect : ScriptableObject {
    public abstract void Apply(CaseData caseData);
}
```

代码说明： ActionCardSO 的结构直观体现了一个行动所需的各部分：基础信息（标题和耗时）、剧情内容（对话步骤）、逻辑约束（条件）和结果影响（效果）。关键部分解析：

- **actionTitle/timeCostHours：** 供UI显示和逻辑使用的基本属性。如 actionTitle 可用于按钮文本或日志，timeCostHours 则用于时间计算（CaseManager.ExecuteAction 会用它加到 currentHour上）。这使得不同行动有不同耗时，策划可以在资产里自由调整而无需改代码。
- **steps 列表 (ActionStep)：** 以 List<ActionStep> 形式序列化在Inspector中。Unity允许对自定义 Serializable类在Inspector直接编辑，因此策划可以展开一个ActionCardSO看到其中steps数组，每个 ActionStep里填写对话人和对白文本，以及可选的玩家选项数组。ActionStep类被标记为 [System.Serializable] ³⁰，包含 speakerName 和 dialogueContent，以及 playerChoices 数组（前面TalkUIHub解释了 PlayerChoice结构）。有了这些，整个对话剧本都存储在资产中。例如在Inspector 中，“Josiah 电话行动”资产的 steps[0] 填入 speakerName="Josiah", dialogueContent="...我已经处理了，我不在城里”，没有playerChoices；steps[1] 可能 speakerName="系统", content="对话出现异常关键词：交易所... 是否再次联系确认?"，包含一个 playerChoice "再次拨打" 设置flag "NeedSecondCall"; steps[2] 再是Josiah的二次通话对白等等。运行时，CaseManager调用 TalkUIHub.StartDialogue，将这个 steps 列表传过去顺序显示。
- **conditions 列表 (Condition)：** 条件采用多态的ScriptableObject设计，允许在编辑器中为每个行动挂多个条件（且关系为且，即必须全部满足才可执行）。Condition是抽象类，实际项目中会有具体子类，比如 FlagCondition（检查某CaseManager全局flag是否为true）、ClueFoundCondition（检查 CaseData某线索是否已获取）等。每个 Condition资产在Inspector配置自身需要的参数（如 FlagCondition有 flagName 属性）。当判断时，CaseManager.GetAvailableActions 调用 Condition.IsMet 来逐一评估 ¹⁶。只有所有条件都返回true，此行动才加入可用列表。若 conditions 列表为空，则表示无前提条件。通过这种设计，策划可以可视化地组合多个条件来限定行动出现。例如给“线下追债”行动添加 ClueFoundCondition(address)，只有拿到地址线索时才允许执行。
- **effects 列表 (Effect)：** 和Condition类似，Effect也是抽象基类，有各种子类实现（UnlockClueEffect, AddMoneyEffect等）。当行动完成时，CaseManager.OnActionFinished 遍历 effects 执行 ¹⁶。每个 Effect 的 Apply 实现会对 CaseData 或全局状态进行修改。例如 UnlockClueEffect(asset配置了线索名“地址”)的 Apply 会设置 caseData.addressFlag=true; AddMoneyEffect(asset上配置金额30000)的 Apply 则调用 MoneyManager.AddMoney(30000)。Effect 让程序逻辑模块化，新增剧情结果只需添加新Effect子类即可，被各Action复用。策划也能直观地为某行动勾选想要的效果，而不用改变脚本。例如在Inspector里给“Josiah 线下追债”行动的 effects 添加一个 AddMoneyEffect(30000) 和一个 CompleteCaseEffect，这样执行该行动后自动收款并结案。ActionCardSO里提供了 markCaseCompleted bool ¹⁶ 作为一种快捷标记，相当于 CompleteCaseEffect的简单情况，主要供程序判定。根据项目需要，可以用Effect体系完全取代这个 bool，用Effect更灵活（因此markCaseCompleted可选，看团队喜好）。
- **CreateAssetMenuAttribute：** 顶部的 [CreateAssetMenu(menuName="Game/ActionCard")] 使得在Unity菜单 Assets/Create/Game 下出现 "ActionCard" 项，方便创建ScriptableObject资产。策划可右键创建 ActionCard 资产，然后在Inspector填数据，不用每次写代码新建类。

Condition/Effect 基类说明： 此处将 Condition 和 Effect 定义为抽象ScriptableObject ³¹ ³²，具体子类需实现 IsMet 和 Apply 方法。这符合策略模式：让数据驱动逻辑判断和执行。例如FlagCondition子类：

```
[CreateAssetMenu(menuName="Game/Conditions/FlagCondition")]
public class FlagCondition : Condition {
    public string flagName;
    public bool requiredValue = true;
    public override bool IsMet(CaseManager caseMgr, CaseData caseData) {
        return caseMgr.GetFlag(flagName) == requiredValue;
    }
}
```

Effect子类例如：

```
[CreateAssetMenu(menuName="Game/Effects/UnlockClue")]
public class UnlockClueEffect : Effect {
    public string clueKey;
    public override void Apply(CaseData caseData) {
        // 将对应线索标记为已解锁（需CaseData有相应字段）
        // 例如：caseData.profile.MarkClueFound(clueKey);
    }
}
```

通过这种体系，我们在 **Inspector 中拖拽** Condition 和 Effect 资产到 ActionCardSO 的列表，实现高度数据驱动的剧情逻辑配置。这在团队合作中非常重要：程序只需提供通用的Condition/Effect脚本，策划便能自由组合实现复杂条件逻辑，而不必请求程序修改专有脚本。例如Josiah案例中新增一个特殊条件“第五天前不可执行”，程序可实现一个 DayCondition（检查CaseManager.currentHour所在天数），策划将其加到Action的conditions即可。

EndingManager.cs — 结局管理器脚本

脚本职责： EndingManager 管理结局场景的演出流程。它在游戏运行过程中监视结局触发条件（或由外部直接调用触发），并在结局场景中负责根据玩家业绩选择相应的文本内容，通过Typewriter效果播放剧情，最后提供退出选项。EndingManager 主要存在于结局场景中（也可在主场景常驻用于监听条件），确保结局演出顺利进行。

```
using UnityEngine;
using TMPro;
using UnityEngine.SceneManagement;

public class EndingManager : MonoBehaviour {
    public static EndingManager Instance { get; private set; }

    [Header("结局文本输出")]
    public TMP_Text endingText; // 用于显示结局文本的TMP_Text
    public GameObject exitButton; // “退出”按钮对象
    [TextArea] public string badEndingContent; // 失败结局文本
    [TextArea] public string normalEndingContent; // 普通结局文本
    [TextArea] public string goodEndingContent; // 成功结局文本

    private bool endingTriggered = false;
```

```

void Awake() {
    if (Instance == null) Instance = this;
}

void Start() {
    // 结局场景加载后自动开始演出
    if (endingText != null) {
        PlayEndingText();
    }
    if (exitButton != null) {
        exitButton.SetActive(false);
    }
}

/// <summary>触发结局演出。如果在主场景调用则负责加载结局场景。</summary>
public void TriggerEnding() {
    if (endingTriggered) return; // 防止重复调用
    endingTriggered = true;
    // 如果尚未在结局场景，则加载之
    if (SceneManager.GetActiveScene().name != "EndingScene") {
        SceneManager.LoadScene("EndingScene");
    } else {
        // 已在结局场景则直接开始演出
        PlayEndingText();
    }
}

/// <summary>根据玩家业绩选择结局文本并播放（逐字显示）。</summary>
private void PlayEndingText() {
    // 决定播放哪个结局内容
    string content;
    if (!CaseManager.Instance.IsAllCasesCompleted()) {
        // 有未完成案件 -> 最终失败
        content = badEndingContent;
    } else if (!MoneyManager.Instance.IsTargetReached()) {
        // 全部完成但未达目标 -> 普通结局
        content = normalEndingContent;
    } else {
        // 全部完成且达标 -> 成功结局
        content = goodEndingContent;
    }
    // 设置文本内容并使用Typewriter效果播放
    endingText.text = content;
    // 若有TypewriterTMP组件，启动打字机效果
    TypewriterTMP typewriter = endingText.GetComponent<TypewriterTMP>();
    if (typewriter != null) {
        typewriter.onComplete += OnTextComplete;
        typewriter.StartTyping(); // 假设该方法开始逐字显示
    } else {
        // 无打字机脚本则直接显示全文并直接显示退出按钮

```

```

        endingText.maxVisibleCharacters = content.Length;
        OnTextComplete();
    }
}

/// <summary>当结局文本播放完毕时调用，显示退出按钮。</summary>
private void OnTextComplete() {
    if (exitButton != null) {
        exitButton.SetActive(true);
    }
}

/// <summary>退出游戏，供退出按钮调用。</summary>
public void QuitGame() {
    Application.Quit();
}
}

```

代码说明： EndingManager 作为结局场景控制脚本，实现了从场景切换到文本演出再到退出的完整流程。

- **单例模式：** Instance 用于让主场景CaseManager或MoneyManager在需要时调用 EndingManager.Instance.TriggerEnding() 来触发结局。这要求EndingManager预先挂在场景中某对象上。可以选择将EndingManager放在主场景的一个常驻对象并勾选DontDestroyOnLoad，让其在加载结局场景后依然存在并控制演出。上例中为了简化，通过静态Instance方式在任一场景都可访问。
- **TriggerEnding():** 核心方法，用于触发结局 33 34。内部用 endingTriggered 防止重复调用。根据当前Scene判断：如果还在主场景，则调用 SceneManager.LoadScene("EndingScene") 切换场景；如果已经在EndingScene（例如EndingManager本来就位于结局场景，也可能通过单例跨场景存在），则直接调用 PlayEndingText() 开始演出。这样设计可以兼容两种使用情形。通常CaseManager.Update中检测到结束条件后调用 TriggerEnding() 来切换场景 17 16。场景加载完成后，新的 EndingManager.Start 会自动调用 PlayEndingText 开始演出。
- **PlayEndingText():** 私有方法，根据游戏最终情况选择对应的结局文本并呈现 5 35。这里通过调用 CaseManager.IsAllCasesCompleted() 和 MoneyManager.IsTargetReached() 来决定：若有案件未完成则判定为“失败结局”，即使用 badEndingContent；如果全部完成但没达标则用 normalEndingContent；若达标则用 goodEndingContent。 23 36 这些字符串内容可以在Inspector直接填写完整的结局台词段落。然后将选定的content赋给 endingText.text，并尝试获取 TypewriterTMP 组件来逐字播放 20 19。 typewriter.onComplete += OnTextComplete 将结尾事件绑定到显示退出按钮的方法 21。调用 typewriter.StartTyping() 开始逐字显示。如果没有 Typewriter组件（比如忘了挂），则直接显示全文并手动调用 OnTextComplete() 显示退出按钮，以免结局卡死。
- **OnTextComplete():** 在结局文本播放完毕时被调用 21。它简单地将 exitButton 对象 SetActive(true) 来显示退出按钮。此按钮应在Inspector中事先连结 EndingManager.QuitGame 方法。需要注意，如果有多段文本需要串联播放，则onComplete事件应该在每段结束时改为触发下一段播放而不是立即显示退出，这里为简化假定结局内容已整理为一个字符串分段统一播放。如果策划要分段甚至多屏呈现，可以扩展EndingManager，让 onComplete 切换内容或使用多个TMP_Text顺序播放。当前实现以单段结局独白为例。

- **QuitGame():** 通过 `Application.Quit()` 退出游戏³⁷。编辑器模式下不会退出，可加逻辑判断。如果还有结算数据需要保存或上传，也可在此处理。该函数已公开，可直接绑定到Button的OnClick，无需额外参数。

使用说明： 在结局场景中放置一个含TMP_Text的UI对象作为剧本显示框，将EndingManager脚本挂在一个空物体或直接挂在该TMP_Text对象上（也行）。在Inspector中：

- 将 endingText 引用指向结局对白的TMP_Text组件；
- exitButton 引用指向“退出”按钮并在Hierarchy中暂时设为隐藏；
- 将 badEndingContent、normalEndingContent、goodEndingContent 填入相应完整文本（可直接从策划文档粘贴三种结局的台词和描述，支持多行 TextArea）。
- 在 exitButton 的 OnClick 列表中添加 EndingManager -> QuitGame()。

如果EndingManager是从主场景常驻的，那在主场景也要有该对象且不要销毁，或在加载EndingScene后通过FindObjectOfType获取EndingManager.Instance。本文采用简易方式假设EndingScene里也有一个EndingManager，这里不用细究，只要能确保TriggerEnding调用顺利让EndingScene和EndingManager执行即可。

通过以上配置，当CaseManager/MoneyManager判定游戏结束并调用TriggerEnding时，EndingScene加载后会自动播放对应结局文本。玩家读完后点击退出，游戏结束。EndingManager将结局相关的逻辑完全封装，便于将来调整不同结局的内容、播放顺序甚至插入CG动画等。同时，它调用CaseManager和MoneyManager的接口获取所需的判定数据，避免了和它们的紧耦合。例如，如果将来引入一个“隐藏完美结局”（比如超额150%完成目标触发），我们可以在EndingManager中新增一个内容字段和判定分支，而不需要更改MoneyManager；MoneyManager只管提供数字，不关心具体阈值与情节，这很好地符合单一职责原则。

项目结构推荐整理

为了方便后续多人协作开发，建议对项目资产和脚本进行清晰的目录划分。根据本项目特点，以下是推荐的目录结构：

- **Assets/**（Unity主资产目录）
- **Scenes/** – 场景文件夹。包含 MainScene（主游戏场景）、EndingScene（结局场景）等场景文件。
- **Scripts/** – 脚本文件夹。按照功能进一步细分：
 - **Scripts/Managers/** – 管理器脚本，如 `CaseManager.cs`、`MoneyManager.cs`、`EndingManager.cs`、`SoundManager.cs` 等。它们大多是单例，全局控制游戏状态或子系统。
 - **Scripts/UI/** – 界面相关脚本，如 `TalkUIHub.cs`（对话UI控制）、`ProfileItemDisplay_TMP.cs`（档案项显示）、`ContactEntryUI.cs`（联系人列表项）以及可能还有 `GameUIManager.cs`（整体UI管理）等。把所有UI交互脚本集中，方便UI工程师查找和修改。
 - **Scripts/Data/** 或 **Scripts/ScriptableObjects/** – 数据定义脚本，包括 `ActionCardSO.cs` 以及 `Condition`、`Effect` 的基类和子类脚本（例如 `FlagCondition.cs`、`UnlockClueEffect.cs`、`AddMoneyEffect.cs` 等）。这些脚本主要用于定义 ScriptableObject 类型，让策划制作资产来配置。也可以进一步把 Conditions/Effects 分入子文件夹：
 - **Scripts/Data/Conditions/** – 各种 Condition 脚本。
 - **Scripts/Data/Effects/** – 各种 Effect 脚本。
 - **Scripts/Models/**（可选）– 如果项目有专门的模型或数据类（例如 ProfileData, CaseData 等纯数据结构），可集中放这里，以示区别于逻辑脚本。本例中 CaseData 定义在 CaseManager.cs 内部，如需独立也可放入Models文件夹。
 - **Scripts/Utilities/**（可选）– 放置一些工具类脚本，如 `CameraFixed.cs`（相机比例修正）或其他通用扩展函数。

- **Resources/**（如果使用）– 存放需要运行时加载的资源。如果 ActionCardSO 等需要在运行时通过名称加载，可以放在 Resources/ActionCards/ 下。但是若通过 Inspector 引用，不强制要求放Resources，放ScriptableObject文件夹下直引用即可。Resources更多用于需要按名字/路径加载的资产。
- **Art/** – 美术资源文件夹，可细分 Textures, Sprites, Prefabs, Materials 等。比如角色头像图片、UI预制体等。UI预制体如选项按钮预制体可以放在 **Art/Prefabs/UI/**。角色小头像放 **Art/Sprites/Portraits/**等。按团队习惯组织美术资源，以免和脚本混杂。
- **Audio/** – 音频资源文件夹。将背景音乐和音效分类存放，例如 **Audio/BGM/** 和 **Audio/SFX/**。在 SoundManager 的 Inspector 可以从这里拖入 AudioClip 或建立音效库。音频策划可以在此管理音效资源，不影响程序逻辑。
- **Fonts/** – 字体资源，如本项目可能使用中文字体给TMP文本。将字体文件和TMP FontAsset放在此方便定位。
- **Plugins/**（如果有）– 第三方插件或库放这，便于和我们自己脚本区分。

采用如上结构后，多人协作时各自负责的内容就有明确归类：

- 程序员撰写的核心逻辑脚本都在 Scripts 子目录，根据模块分类，互不干扰。同时公共的 Data 定义也集中，方便检查。例如要调整某Condition逻辑，可以直接打开 Scripts/Data/Conditions 找到对应脚本。
- 策划配置的资产如 ActionCardSO 则可以集中放在 **Assets/Data/ActionCards/** 目录下，每个资产文件命名为“CaseX_ActionY.asset”等，方便批量查看。Condition和Effect资产也可分类存放（如 **Data/ConditionsAssets/**、**Data/EffectsAssets/** 等），并以有意义的名字命名（如“HasAddress.condition.asset”，“AddMoney30000.effect.asset”）。这样当策划需要复用或修改某种Condition/Effect时，可以很快通过目录找到资产和对应脚本实现。
- UI相关的预制体和素材分门别类：设计 UI 的美术/前端工程师只需关注 Art和Prefabs下的内容和 Scripts/UI 脚本，逻辑代码改动不会破坏UI资源，反之亦然。

总之，上述结构清晰地将**场景、脚本、数据资产、美术资源、音频**等分隔，既减少了版本控制冲突，又使得每类文件各归其位，后续新人加入也能快速理解项目内容。团队可以在此基础上根据实际需要微调（例如加入 **Docs/** 文档文件夹，**Tests/** 测试脚本文件夹等）。关键是**确保同类内容放在一起、命名规范一致**，这样协作开发才能高效有序。

音效系统拓展方案

为增加游戏反馈和氛围，我们扩展一个完整的音效管理播放系统。通过集中管理音效的播放，可以方便地在任意脚本中触发音效，而不用每处反复拖拽AudioSource或AudioClip。实现步骤和方案如下：

1. **创建 SoundManager 单例：** 新建脚本 `SoundManager.cs` 并挂载在主场景一个名为“SoundManager”的空GameObject上。将其设计为全局唯一的单例（类似CaseManager的Instance用法）。SoundManager 内包含一个全局的 AudioSource，用于播放短效音效（Sound Effects, SFX），以及音效资源库。如需背景音乐BGM管理，也可在此扩展，但本次重点是音效。
2. **提供 PlaySFX(string name) 接口：** 在 SoundManager 中编写 `public void PlaySFX(string clipName)` 方法。其实现可以通过查找预先登记的音效列表来找到名称匹配的 AudioClip，然后调用 `AudioSource.PlayOneShot(clip)` 播放。³⁸ 这样，其他脚本只需一行 `SoundManager.Instance.PlaySFX("某音效名")` 即可播放对应声音。通过名字索引音效，使得策划/程

序可以以语义化的标识触发声音，而不需要关心具体AudioClip引用，方便后期替换资源而代码无需改动。

3. **Inspector 配置 AudioSource 与 AudioClip:** 在 SoundManager 脚本中，定义一个 `public AudioSource audioSource;` 和一个 AudioClip 列表（或字典）。可以定义一个辅助类 SoundItem 包含 name 和 AudioClip，然后在 SoundManager 中暴露一个 SoundItem 数组供编辑。 18 39 例如：

```
[System.Serializable]
public class SoundItem {
    public string name;
    public AudioClip clip;
}
public SoundItem[] soundItems;
```

在Inspector中，音频设计人员可以向 soundItems 数组添加元素，每个指定一个音效名字和对应的AudioClip文件。比如 name="Success", clip=（拖入成功音效文件）；name="Fail", clip=失败音效文件；name="ButtonClick", clip=按钮点击音效等。还需要将场景中的 AudioSource 引用赋给 SoundManager.audioSource（可将 AudioSource 组件加在同一个 SoundManager 对象上）。确保 AudioSource 的 Play On Awake 关掉（不然一开始会自动播放空Clip），Spatial Blend 设为2D（因为UI和大部分效果用2D音）。

1. **实现 SoundManager 播放逻辑:** 下面给出 SoundManager 的示例代码：

```
using UnityEngine;
public class SoundManager : MonoBehaviour {
    public static SoundManager Instance { get; private set; }
    public AudioSource audioSource;
    [System.Serializable]
    public class SoundItem {
        public string name;
        public AudioClip clip;
    }
    public SoundItem[] soundItems;
    private Dictionary<string, AudioClip> soundMap;
    void Awake() {
        if (Instance == null) Instance = this;
        // 建立名字到AudioClip的映射字典
        soundMap = new Dictionary<string, AudioClip>();
        foreach (var item in soundItems) {
            if (!soundMap.ContainsKey(item.name)) {
                soundMap.Add(item.name, item.clip);
            }
        }
    }
    public void PlaySFX(string name) {
        if (soundMap != null && soundMap.ContainsKey(name)) {
            AudioClip clip = soundMap[name];
            audioSource.PlayOneShot(clip);
        } else {
```

```

        Debug.LogWarning($"找不到名为{name}的音效");
    }
}
}

```

Awake 中将 soundItems 转为 Dictionary 以便快速查找³⁹。PlaySFX 检查字典中有无该名称，有则调用 PlayOneShot。⁴⁰ 这样设计保证同一时刻可以重叠播放多个音效（PlayOneShot会混音），不会打断之前的声音。

1. **使用方法嵌入 Action 或 UI 事件：** 准备好SoundManager后，就可以在游戏各处触发音效：
2. **在脚本中调用：** 任何脚本只要在发生某事件时需要声音反馈，调用 `SoundManager.Instance.PlaySFX("音效名")`。例如，在 TalkUIHub 显示下一句对白时，可以加入 `SoundManager.Instance.PlaySFX("TalkBeep")`，每当角色对话出现就播放一段提示音。又或者在 MoneyManager 增加金额时播放“coin”音效。通过名字调用，代码更易读，也避免直接引用 AudioClip造成的依赖。
3. **在Unity事件中调用：** 对于UI按钮等，也可以不经脚本，在Inspector中直接绑定 PlaySFX。Unity的 Button.onClick 支持调用无参或单参函数。如果要直接在Button上配置音效，给 SoundManager 加一个重载方法如 `PlaySFX_Button(string name)`（因为Unity事件面板无法直接输入参数给 `PlaySFX(string)`）。不过其实可以做到：在按钮的onClick中拖入 SoundManager 对象，然后选择函数 `SoundManager.PlaySFX`，Unity会显示一个文本框让你输入字符串参数。这是因为PlaySFX签名匹配 `UnityEvent<string>`。因此我们可以直接利用已有的 `PlaySFX(string)`。比如，将Button的onClick添加 `SoundManager->PlaySFX(string)`，在框中填入 "ButtonClick"，就实现了按下该按钮播放点击音效的功能。

示例：我们希望玩家在点击任何选项按钮时有反馈音。可以在 ContactEntryUI 或 TalkUIHub 生成按钮的地方，给每个按钮的 onClick额外添加 `SoundManager.Instance.PlaySFX("Click")`。或者更简单，制作按钮预制体时就绑定 PlaySFX，凡是实例化都自带音效。亦或在UI的EventSystem中用Click声音配置，这里不展开。重要的是，通过SoundManager集中管理音源，我们不用在每个脚本拖一遍AudioSource，也避免了同时播放多个音效时互相干扰的情况。

1. **扩展：背景音乐BGM** – 如果需要背景音乐循环播放，也可以在SoundManager加入一个专门的 AudioSource或AudioSource组件数组，并提供 PlayBGM/PauseBGM 切换音乐的方法。BGM通常一个场景一个，SoundManager可以在切换场景时检查播放适合的BGM。这部分可根据需求添加。当前项目重点在SFX，暂不详述。

通过以上方案，音效系统基本成型。将 SoundManager.cs 脚本放入 **Scripts/Managers** 文件夹，音效资源放入 **Audio/** 文件夹，所有音效的调用都集中通过 SoundManager的接口完成，修改起来非常方便。在团队协作中，美术/音频负责人可以自由地增加音效资源并在SoundManager的soundItems里登记，新加的音效可以告知程序同事使用对应名字来调用即可，无需关心AudioSource细节。程序员如需调整音效逻辑（比如音量统一调节，或播放位置3D效果），也只需改SoundManager一处即可影响全部相关调用。

最后，在调试时请注意检查Console警告：如果 `SoundManager.PlaySFX` 提示“找不到某音效名”，说明名字拼写不一致或未在soundItems里注册。团队应约定统一的音效命名并文档化，比如使用常量或枚举来管理，以减少出错。也可以进一步在 SoundManager启动时自动从 Resources/Audio/SFX 文件夹加载所有clip并以文件名作为key，这样音频人员添加文件就自动可用。但在音效不多情况下，手动列表维护更直观。

综上，音效系统的扩展为游戏增色不少。在完成上述集成后，我们可以很容易地在关键情境加入音效提升反馈：电话拨通音、收到短信提示音、成功追回债款的现金声音效、结局时不同的音乐（在EndingManager中也可通过SoundManager播放不同BGM）等等。这将使玩家体验更加丰富和有沉浸感。

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 18 19 20 21 22 23 24 25 26 27 28 29 30

31 32 33 34 35 36 37 38 39 40 剧情1代.docx

file:///file_0000000009c461f99285a40988404aab

17 CameraFixed.cs

file:///file_0000000005be861f7a4591df487af9647